

Database Comparison Report

Overview

This report compares the MySQL and DynamoDB shopping cart implementations built in `hw8` using the actual performance data collected during the assignment. The comparison uses the merged `combined_results.json` file, which contains 300 total records: 150 from MySQL and 150 from DynamoDB. Each backend was tested with the exact same workload: 50 cart creations, 50 item-update operations, and 50 cart retrievals.

The goal is not to repeat generic SQL-versus-NoSQL advice. Instead, the goal is to decide which database fits this shopping cart implementation based on measured latency, observed consistency behavior, operational complexity, and likely production scenarios.

Measured Results

At the top level, MySQL performed slightly better than DynamoDB in this implementation:

Metric	MySQL	DynamoDB
Avg response time	221.40 ms	232.32 ms
P50	210.58 ms	213.93 ms
P95	302.78 ms	310.23 ms
P99	320.72 ms	358.47 ms
Success rate	100%	100%

MySQL therefore won average latency by `10.92 ms`, p95 by `7.45 ms`, and p99 by `37.75 ms`. Both systems were functionally reliable during the 150-operation test, since both completed all 150 requests successfully.

The operation-level breakdown was more nuanced:

Operation	MySQL Avg	DynamoDB Avg	Winner
Create cart	219.93 ms	248.65 ms	MySQL
Add items	226.18 ms	237.08 ms	MySQL

Operation	MySQL Avg	DynamoDB Avg	Winner
Get cart	218.07 ms	211.24 ms	DynamoDB

DynamoDB was faster only for `GET_CART`, which fits the document-style, key-based retrieval model. MySQL still won the overall result because create and add-item operations were faster, and its tail latency was better.

Consistency and Correctness Findings

The most important lesson from the comparison was not average latency. It was correctness under concurrent updates.

For MySQL, the shopping cart design used a normalized schema and a transactional update approach. That meant row updates and item merges were naturally aligned with the relational model. In practice, the API behaved predictably and there were no special consistency concerns in the test results.

For DynamoDB, the implementation used one table keyed by `cart_id`, with cart items embedded inside each cart document. This worked well for simple key-based reads and writes. In the eventual consistency investigation, immediate read-after-write behavior was acceptable: create-then-get and add-then-get both succeeded on the first read attempt during the test run. However, a more important weakness appeared under concurrent writes to the same cart. Multiple requests could all return success while some item updates were overwritten because the application used a read/modify/write pattern on the full item list.

That finding matters more than the small latency gap. If this exact code were used in production, MySQL would be the safer choice for shopping carts because it better protects correctness in the presence of concurrent edits. DynamoDB can still work for carts, but only if the update strategy is strengthened with conditional writes, version checks, or a different item model.

Resource and Operational Comparison

The two systems also differed sharply in how they consumed resources and how much operational work they required.

MySQL required a full RDS deployment, credentials, schema management, security group wiring, and connection pool configuration. During the load test, the `db.t3.micro` instance remained lightly utilized: CPU averaged roughly `3.94%`, peak connections stayed at `1`, and ECS CPU and memory also remained low. This shows that the assignment-scale workload did not stress the relational setup, but it still required more infrastructure to get right.

DynamoDB was operationally lighter. There was no connection pool to manage and no SQL schema migration step. CloudWatch showed low consumed read and write capacity throughout the test

window, and the table used `PAY_PER_REQUEST`, which avoided manual throughput planning. This made DynamoDB more appealing from a scaling and operations perspective, especially for bursty traffic. The tradeoff was that application-side write correctness became more important than in the MySQL design.

In short:

- MySQL had higher infrastructure overhead but stronger default correctness.
- DynamoDB had lower operational overhead but required more careful application design for concurrent updates.

Scenario Recommendations

For a **Startup MVP**, I would choose **MySQL**. The measured latency was slightly better, and the relational model made cart behavior easier to reason about. For one developer moving quickly, safer default write behavior matters more than theoretical horizontal scale.

For a **Growing Business**, I would still choose **MySQL**. Feature expansion usually increases the need for richer queries, joins, reporting, and stronger transactional behavior. The current MySQL design fits that evolution better than the current DynamoDB cart document approach.

For **High-Traffic Events**, I would choose **DynamoDB**, but only after redesigning the update path. Even though DynamoDB was slower overall in this assignment, its managed scaling model, low consumed capacity, and lack of connection management make it more attractive for sudden spikes than a small single-instance RDS deployment.

For a **Global Platform**, I would also choose **DynamoDB**, again with a stronger concurrency-safe write design. The patterns learned in this assignment suggest that DynamoDB aligns more naturally with globally distributed, always-on, highly elastic systems than the single-region relational setup used here.

Final Recommendation

For the shopping cart implementation actually built in this assignment, **MySQL is the winner**. The recommendation is based on three pieces of evidence:

1. It was faster overall in the measured workload.
2. It had better tail latency.
3. It behaved more safely under the cart update pattern used in the application.

That does not mean DynamoDB is a poor choice. It means DynamoDB was more sensitive to data-model and write-strategy design decisions. If the system requirements changed toward highly bursty traffic, large-scale elasticity, or globally distributed access, DynamoDB would become much more attractive. In a complete e-commerce platform, I would use both: MySQL for order history and other

highly relational workflows, and DynamoDB for session-style or large-scale key-value workloads where managed scaling is the bigger priority.